WIA1002/WIB1002 Data Structure

Individual assignment:

You are required to develop a console-based Grocery Store Management System in Java without any graphical user interface. The system consists of **two independent but interacting modules**. The first module is the **Inventory Management Module**, which handles all product-related operations such as adding new products, searching for products by ID or name, removing products, updating stock quantities, and displaying the entire inventory. For this module, you must use an **ArrayList (the built-in java.util.ArrayList)** as the underlying data structure because inventory requires frequent random access by index, moderate insertion and deletion, and the ability to dynamically resize. The second module is the **Shopping Cart Module,** which manages customer purchases. When a customer adds an item to the cart, the system must check availability in the inventory, reduce the stock temporarily, and store the selected product along with the desired quantity. For the shopping cart, you must use a **Singly Linked List implemented entirely by yourself (not java.util.LinkedList)** because the cart requires sequential access, frequent additions and removals from the front or back, and no random access by index. Additionally, you must **implement a Stack (using your own singly linked list based implementation) to provide an undo feature that allows the customer to remove the last item added to the cart.** At the beginning of program execution, the inventory must be loaded from a text file named **inventory.txt.** At the end of the session, any changes to the inventory must be saved back to the same file. The system must generate a detailed bill at checkout, update the inventory file with final stock quantities, and clear the cart. The entire program must run in a loop with a text-based menu until the user chooses to exit.

Functionalities Required

Task 1: Inventory Management Module

No.	Functionality	Description
1	Load inventory from file	Read inventory.txt (format: ID,Name,Price,Stock) and populate the ArrayList of type items
2	Display all products	Show all products in a formatted table with ID, Name, Price, and Stock
3	Search product by ID	Accept integer ID, return full product details if found
4	Search product by name	Accept string (case-insensitive, partial match allowed), return all matching products
5	Add new product	Accept ID, Name, Price, Stock from user; add to inventory (no duplicate ID allowed)
6	Remove product	Accept ID, remove product from inventory only if it exists
7	Update stock	Accept ID and new stock quantity, update the inventory
8	Save inventory to file	Write current inventory from ArrayList back to inventory.txt (overwrite)

Task 2: Shopping Cart Module

No.	Functionality	Description
9	Add item to cart	Accept product ID and quantity; check stock in inventory; if available, add to cart using singly linked list; temporarily reduce stock in inventory
10	View cart	Display all items in cart with product name, quantity, unit price, and subtotal per item
11	Remove item from cart	Accept product ID, remove that item completely from cart; restore its stock back to inventory
12	Update item quantity in cart	Accept product ID and new quantity; adjust stock in inventory accordingly
13	Clear cart	Remove all items from cart and restore all stock back to inventory
14	Undo last cart addition	undo the most recent addition by removing that item from the cart and restoring its stock in inventory- Stack (Hint: same implementation of Linkedlist can behave as stack with restricted insertion and removal)

Task 3: Billing & Exit

No.	Functionality	Description
15	Generate bill / checkout	Calculate total bill amount; display itemized bill; reduce final stock in inventory permanently; clear cart; clear undo stack; ask user if they want to save inventory
16	Save and exit	Save current inventory to file; exit program

Classes and Data Structures Required

Class 1: Product

Attribute	Type	Description
id	int	Unique product identifier
name	String	Product name
price	double	Price per unit
stock	int	Available quantity in inventory

Methods: Constructor, getters, setters, toString()

Class 2: InventoryManager

Data Structure	java.util.ArrayList<Product>
Method	**Description**
loadFromFile(String filename)	Reads inventory.txt and populates ArrayList
saveToFile(String filename)	Writes current inventory to file
addProduct(Product p)	Adds product to ArrayList (check duplicate ID)
removeProduct(int id)	Removes product by ID
searchById(int id)	Returns product or null
searchByName(String name)	Returns ArrayList of matching products
updateStock(int id, int newStock)	Updates stock quantity
displayAll()	Prints all products in table format
getProductById(int id)	Returns product for cart operations
isAvailable(int id, int requestedQty)	Checks if sufficient stock exists

Class 3: CartNode (For Singly Linked List)

Attribute	Type	Description
product	Product	Reference to product (from inventory)
quantity	int	Quantity added to cart
next	CartNode	Pointer to next node in cart

Class 4: CartList (Self-Implemented Singly Linked List)

Data Structure	**Singly Linked List (custom implementation)**
Method	**Description**
addItem(Product p, int qty)	Adds new node at the end; if product exists, updates quantity
removeItem(int productId)	Removes node with matching product ID
updateQuantity(int productId, int newQty)	Changes quantity of existing cart item
findItem(int productId)	Returns CartNode or null
displayCart()	Prints all cart items with subtotals
calculateTotal()	Returns total bill amount as double
clear()	Empties the entire cart (sets head to null)

undo()	to remove last added item/product
getSize()	Returns number of items in cart
isEmpty()	Checks if cart is empty

Class 6: GroceryStoreSystem (Main Class)

Function	**Description**
Display main menu	Show 10–12 options repeatedly until exit
Accept user input	Use Scanner for all inputs
Call appropriate methods from InventoryManager, CartList	
Handle exceptions	File not found, invalid input, product not found, insufficient stock
Manage program flow	Load inventory at start, save on exit

Task 4: Answer following MCQs

1. **The inventory module in the Grocery Store Management System uses ArrayList<Product> instead of a linked list. What is the primary reason for this choice?**

a. ArrayList uses less memory than a linked list for storing products

b. ArrayList provides O(1) access by index, which is beneficial for random product lookups by ID

c. Linked lists cannot store objects of type Product

d. ArrayList automatically sorts products alphabetically

2. **The shopping cart uses a self-implemented singly linked list rather than java.util.ArrayList. Which of the following best justifies this design decision?**

a. Singly linked lists allow O(1) insertion and deletion at arbitrary positions, which is common in cart operations

b. ArrayList cannot store multiple quantities of the same product

c. The cart requires frequent sequential traversal and insertions/removals at the ends, where linked lists excel, and random access by index is rarely needed

d. Singly linked lists automatically prevent duplicate products in the car

3. **The undo feature uses a stack implemented with a singly linked list but exposes only push() and pop() methods. This design demonstrates which important computer science principle?**

   a. Dynamic polymorphism

   b. A stack is a restricted version of a linked list (LIFO access only)

   c. Inheritance over composition

   d. Garbage collection optimization

4. **When a customer adds an item to the cart, the system pushes a CartAction object onto the undo stack. Later, when the customer requests undo, the system pops the last action and restores the stock. What would happen if the stack was implemented using an ArrayList instead of a linked list for this specific feature?**

   a. The undo feature would stop working completely

   b. Both implementations would work correctly, but the ArrayList might require occasional resizing and shifting of elements, making push/pop less efficient in some cases

   c. The ArrayList would be faster because it uses contiguous memory

   d. The ArrayList cannot store custom objects like CartAction

5. **In the LinkedListStack<T> implementation, the undo() method removes the node from the head of the linked list. Why the head should be chosen as the "top" of the stack rather than the tail?**

   a. Removing from the head of a singly linked list is O(1), while removing from the tail would require O(n) traversal to find the previous node

   b. The head cannot be used because it would reverse the order of operations

   c. Java requires stacks to use the head as the top

   d. The tail is always null in a stack implementation

6. Consider the following operations performed on the shopping cart and undo :

- Add Apple
- Add Banana
- Undo
- Add Orange

After these operations, which items remain in the shopping cart?

a. Apple, Banana, Orange

b. Apple, Orange

c. Banana, Orange

d. Orange only